



G H RAISONI COLLEGE OF ENGINEERING AND MANAGEMENT, PUNE

AN EMPOWERED AUTONOMOUS INSTITUTE, AFFILIATED TO SAVITRIBAI
PHULE PUNE UNIVERSITY, PUNE

Database Design Proposal

Warehouse Inventory and Stock Management Database

Submitted for
TAE-I
in the subject

Database Management System (DBMS)

Submitted By:
Kanchan Rana(P17)

Department of CSE [Data Science]
G H RAISONI COLLEGE OF ENGINEERING AND MANAGEMENT,
PUNE
Academic Year: 2026–2027

Table of Contents

1. Table of Contents	1
2. Database System Title.....	4
3. Problem Statement & System Requirements	4
3.1 Business Domain	4
3.2 Enterprise Entities	4
3.3 Operational Transactions Supported.....	5
3.4 Business Rules & Integrity Constraints.....	6
3.5 Assumptions.....	6
4. Literature & System Reference	7
4.1 Reference 1.....	7
4.2 Reference 2.....	8
4.3 Reference 3.....	8
5. Conceptual Schema Design (ER / EER Diagram)	9
5.1 Entities, Attributes, and Keys.....	10
5.2 Relationships, Cardinality Ratios, and Participation Constraints	10
6. Relational Schema & Normalisation Plan	11
6.1 Relational Schema.....	11
CATEGORY	12
SUPPLIER.....	12
PRODUCT.....	12
WAREHOUSE	13
EMPLOYEE.....	13
CUSTOMER	14
PURCHASE_ORDER	14
PURCHASE_ORDER_DETAIL.....	15
SALES_ORDER	15
SALES_ORDER_DETAIL	16
INVENTORY	16
SHIPMENT	17
STOCK_MOVEMENT	18
6.2 Normalisation Demonstration.....	18

6.2.1 Unnormalised Form (UNF)	19
6.2.2 First Normal Form (1NF)	20
6.2.3 Second Normal Form (2NF)	20
6.2.4 Third Normal Form (3NF)	21
6.2.5 Boyce-Codd Normal Form (BCNF)	22
6.3 Referential Integrity Summary	22
7. Target RDBMS Software	22
7.1 Justification	23
7.2 Phase II Roadmap	23

2. Database System Title

Warehouse Inventory and Stock Management Database

This project designs a relational database that automates the day-to-day operations of a multi-warehouse distribution enterprise — from supplier procurement and stock receipt to order fulfilment and customer dispatch — while maintaining accurate, real-time inventory records across all facilities.

3. Problem Statement & System Requirements

3.1 Business Domain

Modern distribution and retail-supply enterprises operate one or more warehouses that continuously receive goods from suppliers and dispatch goods to customers. Manual or spreadsheet-based inventory tracking in such an environment is error-prone: stock counts fall out of sync with actual shelf quantities, reorder points are missed, and there is no single source of truth linking a sale or purchase to the resulting change in stock. The proposed database formalises these operations into a relational schema so that stock levels, procurement, and sales activity remain consistent, auditable, and query-able at any point in time.

The system is designed for a mid-sized enterprise that operates multiple warehouses in different geographic locations, sources products from a network of suppliers, and sells to both retail and wholesale customers through a dedicated sales workforce.

3.2 Enterprise Entities

The database captures the following core enterprise entities, each representing a distinct object of interest to the business:

Warehouse: A physical storage facility with a location, storage capacity, and an assigned manager.

Product: A stock-keeping unit (SKU) that is bought from suppliers and sold to customers, classified under a Category.

Category: A classification group (e.g., Electronics, Stationery, Perishables) used to organise the product catalogue.

Supplier: An external vendor from whom products are procured through purchase orders.

Customer: An external buyer (retail or wholesale) who places sales orders.

Employee: Warehouse staff, procurement officers, and sales personnel who operate the system.

Purchase Order: A procurement transaction raised on a supplier for one or more products.

Sales Order: A sales transaction committed to a customer for one or more products.

Inventory: The live stock position of every product at every warehouse.

Shipment: The physical movement record of an inbound (supplier → warehouse) or outbound (warehouse → customer) consignment.

Stock Movement: An immutable audit trail of every quantity change against a product at a warehouse.

3.3 Operational Transactions Supported

The database must support the following core business transactions:

- Onboarding new suppliers, customers, products, categories, employees, and warehouses.
- Raising a purchase order against a supplier, specifying the products and quantities required.
- Recording receipt of goods against a purchase order, which increases inventory at the receiving warehouse and generates a stock movement (IN) record.
- Accepting a sales order from a customer, specifying products and quantities to be sold.
- Dispatching goods against a sales order, which decreases inventory at the source warehouse and generates a stock movement (OUT) record.
- Recording inter-warehouse stock transfers as paired OUT/IN stock movements.
- Tracking inbound shipments (from suppliers) and outbound shipments (to customers), including carrier and tracking details.
- Generating low-stock alerts by comparing `Inventory.QuantityOnHand` against `Product.ReorderLevel`.
- Producing supplier performance, sales performance, and warehouse utilisation reports.

3.4 Business Rules & Integrity Constraints

- Every product must belong to exactly one category; a category may have zero or more products.
- A product may be stocked at zero or more warehouses, and a warehouse may stock zero or more products — this many-to-many relationship is resolved through the Inventory entity.
- Every employee is assigned to exactly one warehouse; a warehouse employs zero or more employees.
- A warehouse has at most one manager, and an employee may manage at most one warehouse (1:1, optional).
- A purchase order must reference exactly one supplier, one placing employee, and one receiving warehouse; it must contain at least one product line.
- A sales order must reference exactly one customer, one handling employee, and one fulfilling warehouse; it must contain at least one product line.
- Inventory.QuantityOnHand can never be negative (enforced via CHECK constraint); a sales order dispatch must not be confirmed if it would drive quantity on hand below zero.
- A shipment is either Inbound (linked to a purchase order) or Outbound (linked to a sales order), never both — enforced through a discriminator attribute and a CHECK constraint.
- Every stock movement must reference a valid product, warehouse, and the employee who logged it, and must carry a non-zero signed quantity.
- Referential integrity is enforced on all foreign keys; deletion of a referenced row (e.g., a Product with existing order history) is restricted (ON DELETE RESTRICT) to preserve transaction history.

3.5 Assumptions

- The enterprise operates in a single currency and a single time zone; multi-currency support is out of scope for Phase I.
- Each product is uniquely identified by an SKU code in addition to its surrogate primary key.
- Prices captured on order-detail lines represent the price effective at the time of the transaction and are independent of later changes to Product.UnitPrice.

- Warehouse capacity is expressed in an abstract unit (pallet positions) and is not decomposed into individual bin/rack locations in this phase.
- User authentication, role-based access control, and application-layer logic are outside the scope of this database design and will be addressed in the front-end/middleware layer during implementation.

4. Literature & System Reference

Three sources — spanning a peer-reviewed conference-style research paper, a professional case-study, and standard technical documentation — were reviewed to ground the schema design proposed in this report in established practice.

4.1 Reference 1

Title: APURTI: An Intelligent Warehouse Management System — Optimizing Space Utilization for Enhanced Efficiency and Productivity

Authors: E. Pandey, S. Thorat, D. Chavan, S. Gudhekar (Guide: R. Morey)

Publication Year: 2025

Source: Journal of Emerging Technologies and Innovative Research (JETIR), Vol. 12, Issue 4, ISSN 2349-5162

Summary:

This paper proposes a warehouse management system built around a relational database implemented on MySQL or PostgreSQL, with tables covering products, suppliers, customers, orders, warehouse locations, and stock transactions. The authors emphasise applying indexing and normalisation to the schema to keep data retrieval efficient, and they describe a layered architecture in which the database sits beneath application and presentation tiers. The study also highlights operational gains — inventory accuracy improving substantially and the system sustaining a significant rise in transaction volume — once RFID-driven data capture was paired with a properly normalised backend.

Relevance to the Proposed System:

The paper's proposed table set (products, suppliers, customers, orders, warehouse locations, transactions) closely parallels the entity list adopted in Section 3.2 of this report, and its explicit recommendation of MySQL/PostgreSQL with normalisation and indexing directly supports the RDBMS choice and the normalisation exercise carried out in Sections 6 and 7.

4.2 Reference 2

Title: Relational Database Architecture — Inventory Control System for E-commerce

Authors: Kiel Dang

Publication Year: 2023

Source: Medium (Professional case-study / technical article)

Summary:

This case study walks through designing a relational inventory-control system for a multi-warehouse retailer, starting from business-rule elicitation before any Entity-Relationship Diagram is drawn. It identifies Orders, Warehouses, and Suppliers as the pivotal entities around which purchasing staff and management reporting revolve, and argues that a well-designed inventory schema directly reduces storage cost by making stock-out and reorder decisions visible in the data itself. The article stresses that business rules — not just entities — must be settled first, since they determine cardinalities and participation constraints in the resulting diagram.

Relevance to the Proposed System:

The case study's design sequence (business rules → entities → ERD) is the same methodology followed in Sections 3 and 5 of this report, and its identification of Orders/Warehouses/Suppliers as pivotal entities reinforces the decision to model Purchase Order, Sales Order, Warehouse, and Inventory as first-class entities with explicit associative tables rather than flattened order records.

4.3 Reference 3

Title: How to Design ER Diagrams for Inventory and Warehouse Management

Authors: GeeksforGeeks Editorial Team

Publication Year: 2024

Source: GeeksforGeeks — Standard Technical Documentation ([geeksforgeeks.org](https://www.geeksforgeeks.org))

Summary:

This technical reference documents standard practice for constructing ER diagrams in the inventory-and-warehouse domain, covering entity and attribute identification, primary/foreign key placement, and correct use of cardinality (1:1, 1:N, M:N) and participation notation. It stresses four design qualities that a warehouse ERD should

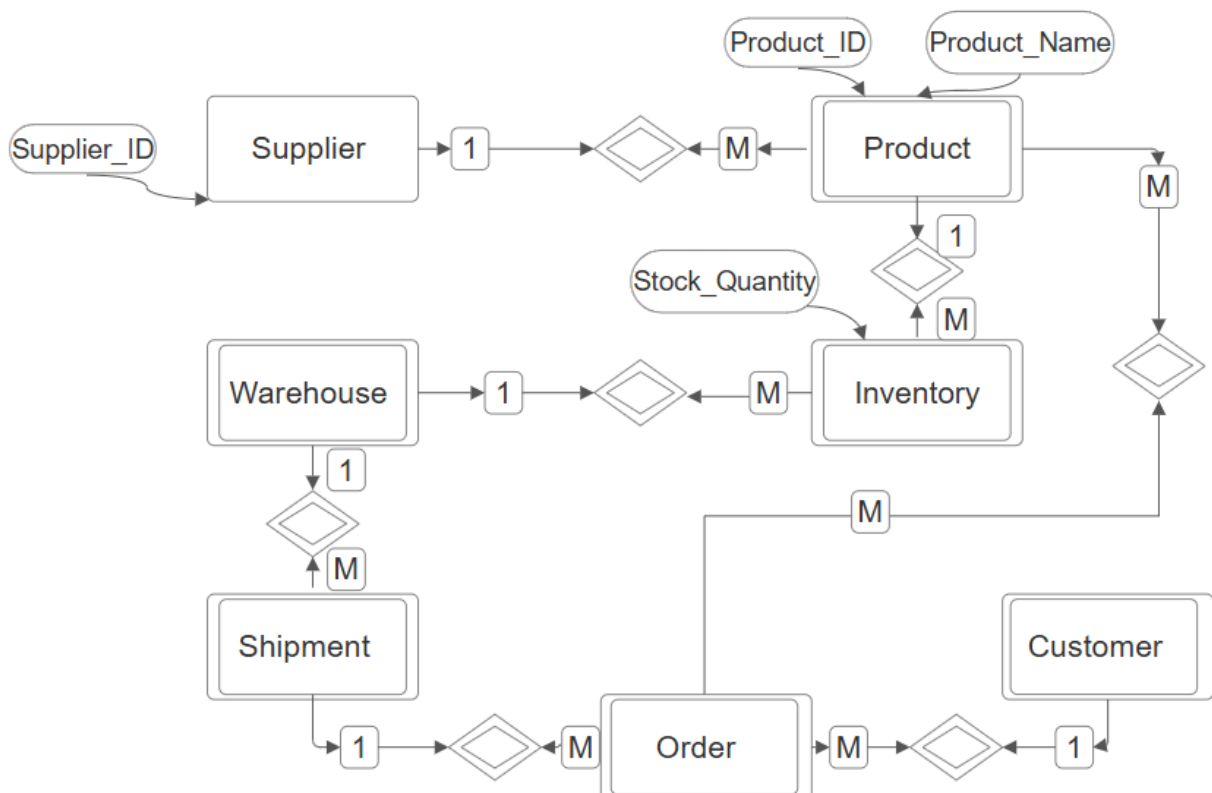
satisfy: readability, correct representation of cardinality and participants, scalability of the resulting schema, and consistency with established relational-modelling conventions.

Relevance to the Proposed System:

The four design qualities named in this reference — readability, correct cardinality/participation representation, scalability, and consistency — were used as the acceptance criteria for the ER/EER diagram produced in Section 5 of this report, particularly in choosing crow's-foot notation and in resolving the Warehouse–Product many-to-many relationship through an explicit Inventory associative entity.

5. Conceptual Schema Design (ER / EER Diagram)

The conceptual schema below models thirteen entities. Regular entities (Category, Supplier, Product, Warehouse, Employee, Customer, Purchase_Order, Sales_Order, Shipment, Stock_Movement) each carry a single-attribute surrogate primary key. Three associative / weak entities — Inventory, Purchase_Order_Detail, and Sales_Order_Detail — carry composite primary keys formed from the foreign keys of the entities they connect, and are used specifically to resolve many-to-many relationships into a set of binary one-to-many relationships, in keeping with relational modelling best practice.



5.1 Entities, Attributes, and Keys

A full attribute-level listing of every entity — including primary keys, foreign keys, and composite keys — is provided in Section 6.1, where each entity is restated as a relational table. The diagram in Figure 5.1 uses the following conventions: an underlined attribute name denotes the primary key (or part of a composite primary key); an italicised attribute name marks a foreign key; entities shaded in orange (Inventory, Purchase_Order_Detail, Sales_Order_Detail) are associative/weak entities whose primary key is wholly or partly composite.

5.2 Relationships, Cardinality Ratios, and Participation Constraints

#	Relationship	Cardinality	Participation
1	Category classifies Product	1 : N	Category: partial · Product: total
2	Supplier is issued Purchase_Order	1 : N	Supplier: partial · Purchase_Order: total
3	Employee places Purchase_Order	1 : N	Employee: partial · Purchase_Order: total
4	Warehouse receives Purchase_Order	1 : N	Warehouse: partial · Purchase_Order: total
5	Purchase_Order contains Purchase_Order_Detail	1 : N	Purchase_Order: total · Detail: total
6	Product is ordered as Purchase_Order_Detail	1 : N	Product: partial · Detail: total
7	Customer places Sales_Order	1 : N	Customer: partial · Sales_Order: total
8	Employee handles Sales_Order	1 : N	Employee: partial · Sales_Order: total
9	Warehouse fulfils Sales_Order	1 : N	Warehouse: partial · Sales_Order: total
10	Sales_Order contains Sales_Order_Detail	1 : N	Sales_Order: total · Detail: total
11	Product is sold as Sales_Order_Detail	1 : N	Product: partial · Detail: total
12	Warehouse employs Employee	1 : N	Warehouse: partial · Employee: total
13	Employee manages Warehouse	1 : 1 (optional)	Employee: partial · Warehouse: partial

#	Relationship	Cardinality	Participation
14	Warehouse — Product stocked as Inventory	M : N (resolved)	Warehouse: partial · Product: partial
15	Purchase_Order generates Shipment (inbound)	1 : N (optional)	Purchase_Order: partial · Shipment: total
16	Sales_Order generates Shipment (outbound)	1 : N (optional)	Sales_Order: partial · Shipment: total
17	Product is tracked by Stock_Movement	1 : N	Product: partial · Stock_Movement: total
18	Warehouse records Stock_Movement	1 : N	Warehouse: partial · Stock_Movement: total
19	Employee logs Stock_Movement	1 : N	Employee: partial · Stock_Movement: total

Note: "Total participation" indicates that every instance of the entity must participate in the relationship (existence dependency); "partial participation" indicates that participation is optional. Relationship 13 (Employee manages Warehouse) is modelled as a separate, optional one-to-one relationship distinct from relationship 12 (Warehouse employs Employee), since a manager is drawn from the pool of employees but not every employee is a manager, and not every warehouse currently has an appointed manager.

6. Relational Schema & Normalisation Plan

6.1 Relational Schema

The ER/EER diagram in Section 5 is mapped to thirteen relational tables as follows. Regular strong entities become tables with a single-column surrogate primary key; each 1:N relationship is implemented by posting the primary key of the "1" side as a foreign key in the "N" side; each M:N relationship (Warehouse–Product, Purchase_Order–Product, Sales_Order–Product) is implemented as a separate bridge table with a composite primary key.

CATEGORY

Stores the classification hierarchy under which products are grouped for cataloguing and reporting.

Attribute	Data Type	Key	Constraint / Notes
CategoryID	INT	PK	AUTO_INCREMENT, NOT NULL
CategoryName	VARCHAR(60)	—	NOT NULL, UNIQUE
Description	VARCHAR(255)	—	NULL

SUPPLIER

Master record of vendors from whom the warehouse procures goods.

Attribute	Data Type	Key	Constraint / Notes
SupplierID	INT	PK	AUTO_INCREMENT, NOT NULL
SupplierName	VARCHAR(100)	—	NOT NULL
ContactPerson	VARCHAR(80)	—	NULL
Phone	VARCHAR(15)	—	NOT NULL
Email	VARCHAR(100)	—	UNIQUE
Address	VARCHAR(200)	—	NULL
GSTNumber	VARCHAR(20)	—	UNIQUE, NOT NULL

PRODUCT

Master catalogue of every stock-keeping unit (SKU) handled by the enterprise.

Attribute	Data Type	Key	Constraint / Notes
ProductID	INT	PK	AUTO_INCREMENT, NOT NULL
ProductName	VARCHAR(120)	—	NOT NULL
SKU	VARCHAR(30)	—	UNIQUE, NOT NULL
CategoryID	INT	FK → Category(CategoryID)	NOT NULL
UnitPrice	DECIMAL(10,2)	—	NOT NULL, CHECK (UnitPrice >= 0)

Attribute	Data Type	Key	Constraint / Notes
ReorderLevel	INT	—	DEFAULT 10, CHECK (ReorderLevel ≥ 0)
UnitOfMeasure	VARCHAR(15)	—	NOT NULL (e.g., PCS, BOX, KG)

WAREHOUSE

Represents each physical storage facility owned or leased by the enterprise.

Attribute	Data Type	Key	Constraint / Notes
WarehouseID	INT	PK	AUTO_INCREMENT, NOT NULL
WarehouseName	VARCHAR(80)	—	NOT NULL
Location	VARCHAR(150)	—	NOT NULL
Capacity	INT	—	CHECK (Capacity > 0), unit: pallets
ManagerID	INT	FK → Employee(EmployeeID)	UNIQUE, NULL (enforces 1:1 with Employee)

EMPLOYEE

Personnel who operate warehouse, procurement and sales functions.

Attribute	Data Type	Key	Constraint / Notes
EmployeeID	INT	PK	AUTO_INCREMENT, NOT NULL
FullName	VARCHAR(100)	—	NOT NULL
Designation	VARCHAR(50)	—	NOT NULL
Phone	VARCHAR(15)	—	NOT NULL
Email	VARCHAR(100)	—	UNIQUE
WarehouseID	INT	FK → Warehouse(WarehouseID)	NOT NULL

Attribute	Data Type	Key	Constraint / Notes
Salary	DECIMAL(10,2)	—	CHECK (Salary > 0)
HireDate	DATE	—	NOT NULL

CUSTOMER

Buyers who place sales orders against warehouse stock.

Attribute	Data Type	Key	Constraint / Notes
CustomerID	INT	PK	AUTO_INCREMENT, NOT NULL
CustomerName	VARCHAR(100)	—	NOT NULL
Phone	VARCHAR(15)	—	NOT NULL
Email	VARCHAR(100)	—	UNIQUE
Address	VARCHAR(200)	—	NOT NULL
CustomerType	ENUM('Retail','Wholesale')	—	DEFAULT 'Retail'

PURCHASE_ORDER

Header record of a procurement order raised on a supplier.

Attribute	Data Type	Key	Constraint / Notes
PurchaseOrderID	INT	PK	AUTO_INCREMENT, NOT NULL
SupplierID	INT	FK → Supplier(SupplierID)	NOT NULL
EmployeeID	INT	FK → Employee(EmployeeID)	NOT NULL
WarehouseID	INT	FK → Warehouse(WarehouseID)	NOT NULL
OrderDate	DATE	—	NOT NULL, DEFAULT

Attribute	Data Type	Key	Constraint / Notes
			CURRENT_DATE
ExpectedDeliveryDate	DATE	—	NULL
Status	ENUM('Pending','Approved','Received','Cancelled')	—	DEFAULT 'Pending'

PURCHASE_ORDER_DETAIL

Line items (products and quantities) associated with a purchase order — resolves the M:N relationship between Purchase Order and Product.

Attribute	Data Type	Key	Constraint / Notes
PurchaseOrderID	INT	PK, FK → Purchase_Order	NOT NULL
ProductID	INT	PK, FK → Product	NOT NULL
QuantityOrdered	INT	—	NOT NULL, CHECK (QuantityOrdered > 0)
UnitCost	DECIMAL(10,2)	—	NOT NULL, CHECK (UnitCost >= 0)

SALES_ORDER

Header record of a sale committed to a customer.

Attribute	Data Type	Key	Constraint / Notes
SalesOrderID	INT	PK	AUTO_INCREMENT, NOT NULL
CustomerID	INT	FK → Customer(CustomerID)	NOT NULL
EmployeeID	INT	FK → Employee(EmployeeID)	NOT NULL

Attribute	Data Type	Key	Constraint / Notes
WarehouseID	INT	FK → Warehouse(WarehouseID)	NOT NULL
OrderDate	DATE	—	NOT NULL, DEFAULT CURRENT_DATE
Delivery Date	DATE	—	NULL
Status	ENUM('Pending','Shipped','Delivered','Cancelled')	—	DEFAULT 'Pending'

SALES_ORDER_DETAIL

Line items (products and quantities) associated with a sales order — resolves the M:N relationship between Sales Order and Product.

Attribute	Data Type	Key	Constraint / Notes
SalesOrderID	INT	PK, FK → Sales_Order	NOT NULL
ProductID	INT	PK, FK → Product	NOT NULL
QuantitySold	INT	—	NOT NULL, CHECK (QuantitySold > 0)
UnitPrice	DECIMAL(10,2)	—	NOT NULL (price at time of sale)

INVENTORY

Associative entity resolving the M:N relationship between Warehouse and Product; records live stock levels per warehouse.

Attribute	Data Type	Key	Constraint / Notes
WarehouseID	INT	PK, FK → Warehouse	NOT NULL
ProductID	INT	PK, FK → Product	NOT NULL

Attribute	Data Type	Key	Constraint / Notes
QuantityOnHand	INT	—	NOT NULL, DEFAULT 0, CHECK (QuantityOnHand >= 0)
LastUpdated	TIMESTAMP	—	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

SHIPMENT

Tracks inbound consignments from suppliers and outbound deliveries to customers (EER specialisation of a generic dispatch record).

Attribute	Data Type	Key	Constraint / Notes
ShipmentID	INT	PK	AUTO_INCREMENT, NOT NULL
ShipmentType	ENUM('Inbound','Outbound')	—	NOT NULL (discriminator attribute)
PurchaseOrderID	INT	FK → Purchase_Order	NULL (set only when ShipmentType = 'Inbound')
SalesOrderID	INT	FK → Sales_Order	NULL (set only when ShipmentType = 'Outbound')
ShipmentDate	DATE	—	NOT NULL
CarrierName	VARCHAR(80)	—	NULL
TrackingNumber	VARCHAR(40)	—	UNIQUE
Status	ENUM('In-Transit','Delivered','Delayed')	—	DEFAULT 'In-Transit'

STOCK_MOVEMENT

Immutable audit log of every quantity change in the warehouse (receipts, dispatches, transfers, and manual adjustments).

Attribute	Data Type	Key	Constraint / Notes
StockMovementID	INT	PK	AUTO_INCREMENT, NOT NULL
ProductID	INT	FK → Product	NOT NULL
WarehouseID	INT	FK → Warehouse	NOT NULL
EmployeeID	INT	FK → Employee	NOT NULL
MovementType	ENUM('IN','OUT','TRANSFER','ADJUSTMENT')	—	NOT NULL
Quantity	INT	—	NOT NULL, CHECK (Quantity > 0)
MovementDate	TIMESTAMP	—	DEFAULT CURRENT_TIMESTAMP
ReferenceType	VARCHAR(30)	—	NULL (e.g., 'PurchaseOrder', 'SalesOrder')
ReferenceID	INT	—	NULL (ID of the originating document)

6.2 Normalisation Demonstration

Normalisation is demonstrated end-to-end on the Sales Order subsystem — the most representative transaction in the database, since it draws together Customer, Employee, Warehouse, Product, and order-line data. The same reasoning applies symmetrically to the Purchase Order subsystem.

6.2.1 Unnormalised Form (UNF)

A naive "flat" sales report, as it might first arrive from a business user as a single spreadsheet row per order, embeds a repeating group of product lines inside each order row:

Sales Order ID	Order Date	Customer ID	Customer Name	Customer Address	Employee ID	Employee Name	Warehouse ID	Warehouse Name	Products (repeating group)
SO101	05-Jul-2026	C01	Rohan Traders	Pune, MH	E01	Aditi Sharma	W01	Pune Central WH	{ P01, LED Bulb 9W, ₹120, Qty 50 }, { P02, USB Cable 1m, ₹90, Qty 30 }
SO102	06-Jul-2026	C02	Metro Retail Hub	Mumbai, MH	E02	Sameer Kulkar ni	W02	Mumbai East WH	{ P03, A4 Paper Ream, ₹230, Qty 20 }

This layout violates First Normal Form because the "Products" attribute is not atomic — it holds a variable-length list of (ProductID, ProductName, UnitPrice, Quantity) tuples within a single cell.

6.2.2 First Normal Form (1NF)

Eliminating the repeating group by giving every product line its own row makes all attribute values atomic. The row is now uniquely identified by the composite (SalesOrderID, ProductID):

Sales OrderID *	ProductID *	Order Date	CustomerID	Customer Name	Customer Address	EmployeeID	Employee Name	WarehouseID	Warehouse Name	Product Name	Unit Price	Qty Sold
SO101	P01	05-Jul-2026	C01	Rohan Traders	Pune, MH	E01	Aditi Sharma	W01	Pune Central WH	LED Bulb 9W	120	50
SO101	P02	05-Jul-2026	C01	Rohan Traders	Pune, MH	E01	Aditi Sharma	W01	Pune Central WH	USB Cable 1m	90	30
SO102	P03	06-Jul-2026	C02	Metro Retail Hub	Mumbai, MH	E02	Sameer Kulkarni	W02	Mumbai East WH	A4 Paper Ream	230	20

(* = composite primary key). All attributes are now atomic and single-valued, so the table satisfies 1NF. However, heavy redundancy remains: order-level attributes (OrderDate, Customer*, Employee*, Warehouse*) repeat on every product line of the same order, and product-level attributes (ProductName, UnitPrice) repeat on every order that includes that product.

6.2.3 Second Normal Form (2NF)

2NF requires that every non-key attribute be fully functionally dependent on the whole primary key, not just part of it. Here the primary key is (SalesOrderID, ProductID). Examining the functional dependencies:

- OrderDate, CustomerID, CustomerName, CustomerAddress, EmployeeID, EmployeeName, WarehouseID, WarehouseName depend only on SalesOrderID — a partial dependency, since ProductID is not needed to determine them.
- ProductName, UnitPrice depend only on ProductID — likewise a partial dependency on the key.

- QuantitySold genuinely depends on the combination of SalesOrderID and ProductID (the quantity sold is specific to that product on that order) — a full dependency on the whole key.

Removing the partially-dependent attributes into tables keyed on the relevant part of the key eliminates this anomaly:

- SALES_ORDER_HEADER (SalesOrderID PK, OrderDate, CustomerID, CustomerName, CustomerAddress, EmployeeID, EmployeeName, WarehouseID, WarehouseName)
- PRODUCT (ProductID PK, ProductName, UnitPrice)
- SALES_ORDER_DETAIL (SalesOrderID FK, ProductID FK, QuantitySold) — composite PK (SalesOrderID, ProductID)

Every non-key attribute in each of the three resulting tables is now fully dependent on that table's entire primary key, so the design satisfies 2NF.

6.2.4 Third Normal Form (3NF)

3NF additionally forbids transitive dependencies — a non-key attribute depending on another non-key attribute rather than directly on the primary key. In SALES_ORDER_HEADER above, SalesOrderID → CustomerID → CustomerName/CustomerAddress is a transitive dependency (CustomerName and CustomerAddress depend on CustomerID, not directly on SalesOrderID); the same is true for EmployeeID → EmployeeName and WarehouseID → WarehouseName. These are removed into their own entities:

- SALES_ORDER (SalesOrderID PK, OrderDate, CustomerID FK, EmployeeID FK, WarehouseID FK, DeliveryDate, Status)
- CUSTOMER (CustomerID PK, CustomerName, CustomerAddress, Phone, Email, CustomerType)
- EMPLOYEE (EmployeeID PK, EmployeeName, ...)
- WAREHOUSE (WarehouseID PK, WarehouseName, ...)
- PRODUCT (ProductID PK, ProductName, UnitPrice, ...)
- SALES_ORDER_DETAIL (SalesOrderID FK, ProductID FK, QuantitySold, UnitPrice) — composite PK

With every transitive dependency removed, each non-key attribute now depends directly, and only, on its own table's primary key — satisfying 3NF. This is precisely the table set already presented in Section 6.1.

6.2.5 Boyce-Codd Normal Form (BCNF)

BCNF requires that for every non-trivial functional dependency $X \rightarrow Y$ in a table, X must be a candidate key (a stricter condition than 3NF, which allows a non-key determinant provided the dependent attribute is itself prime). Examining each 3NF table produced above:

- **SALES_ORDER**: the only determinant of any non-key attribute is SalesOrderID itself, which is the (sole) candidate key.
- **SALES_ORDER_DETAIL**: the only non-key attribute (QuantitySold, UnitPrice) is determined by the full composite key (SalesOrderID, ProductID); no proper subset of the key determines it, and no other determinant exists.
- **CUSTOMER, EMPLOYEE, WAREHOUSE, PRODUCT**: each has a single candidate key (its surrogate ID) and no other determinant appears among the non-key attributes.

Since every determinant in every resulting table is a candidate key, no further decomposition is required — the schema is already in BCNF. The same UNF $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF $\rightarrow$ BCNF reasoning applies identically to the Purchase Order subsystem (Purchase_Order, Purchase_Order_Detail, Supplier) and yields the equivalent table structures already listed in Section 6.1.

6.3 Referential Integrity Summary

All foreign keys in the schema are enforced with ON UPDATE CASCADE / ON DELETE RESTRICT semantics by default, except Warehouse.ManagerID, which is nullable and set ON DELETE SET NULL, allowing an employee to be removed from a managerial role without forcing deletion of the warehouse record. Because Warehouse.ManagerID references Employee while Employee.WarehouseID references Warehouse, the two tables share a circular foreign-key dependency; this is resolved operationally by creating both tables with ManagerID initially nullable, inserting Warehouse rows first (ManagerID = NULL), inserting Employee rows referencing the warehouse, and finally issuing an ALTER TABLE / UPDATE statement to populate ManagerID.

7. Target RDBMS Software

MySQL (Community Edition, version 8.x) is selected as the target relational database management system for the Phase II implementation of the Warehouse Inventory and Stock Management Database.

7.1 Justification

Open-source and zero licensing cost: MySQL is free and open-source under the GPL, which suits an academic project and keeps the total cost of ownership low if the design were carried into a real small/mid-sized enterprise deployment.

Strong ACID compliance with InnoDB: The InnoDB storage engine provides full ACID transaction support, row-level locking, and foreign-key enforcement — all essential for concurrent purchase-order receipts and sales-order dispatches that must atomically update Inventory and Stock_Movement together.

Native support for all constructs used in this design: MySQL natively supports AUTO_INCREMENT surrogate keys, composite primary keys, ENUM types (used for Status/Type discriminator columns), CHECK constraints (MySQL 8.0.16+), and cascading referential actions — every construct specified in Section 6.1 maps directly onto MySQL DDL without workarounds.

Mature indexing and query optimisation: B-tree indexing on primary/foreign keys and composite indexes on bridge tables (Inventory, Purchase_Order_Detail, Sales_Order_Detail) allow the low-stock, reorder, and reporting queries described in Section 3.3 to execute efficiently even as transaction volume grows, consistent with the indexing/normalisation recommendation found in Reference 1 (Section 4.1).

Widespread tooling and ecosystem: MySQL Workbench provides built-in reverse-engineering of ER diagrams from a live schema, which will assist verification of the Phase II implementation against the conceptual design in Section 5. Broad driver support (JDBC/ODBC, Python, Node.js, PHP) also keeps the eventual application layer unconstrained in technology choice.

Proven fit for this problem domain: Both the WMS research paper reviewed in Section 4.1 and the inventory-system case study in Section 4.2 independently converged on MySQL/relational engines for warehouse and inventory workloads, corroborating its suitability here.

7.2 Phase II Roadmap

- Translate the table definitions in Section 6.1 into MySQL DDL (CREATE TABLE statements with PRIMARY KEY, FOREIGN KEY, CHECK, and DEFAULT clauses).
- Populate each table with representative sample data covering all entities and relationships described in Section 5.2.

- Implement the operational transactions listed in Section 3.3 as parameterised SQL statements / stored procedures (e.g., a ReceiveGoods procedure that inserts a Stock_Movement row and updates Inventory.QuantityOnHand within a single transaction).
- Create supporting indexes on frequently filtered/joined columns (ProductID, WarehouseID, SupplierID, CustomerID, OrderDate).
- Build validation queries for referential integrity, low-stock alerts, and supplier/sales performance reports.

8.Dataset Link: https://github.com/eyowhite/Messy-dataset/blob/main/warehouse_messy_data.csv